SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

Question Count: 5

Code of Conduct

I Santhosh kumar.M(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ___Santhosh_____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis

Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Error Analysis Task

1. Hadoop Job Fails Because Input Splits Are Uneven

Problem Analysis

A Hadoop job repeatedly fails or performs poorly because the input data is divided into uneven-sized input splits. In Hadoop MapReduce, input data is divided into smaller portions called **input splits**, which are processed by Mapper tasks. Ideally, the data should be distributed evenly so that all Mapper tasks receive a similar amount of work.

If one input split contains much more data than the others, its Mapper will take considerably longer to complete. Other Mappers may finish early and remain idle. This creates a **data skew** or workload imbalance problem. When the Mapper output is sent to reducers, some reducers may also receive more data than others.

Likely Design Error

The likely design errors are:

- Uneven input split sizes.
- Poor partitioning of input data.
- Very large files mixed with many small files.
- Improper block or split-size configuration.
- Skewed data distribution.
- Poorly designed partitioning logic.

Effects

- Some Mapper tasks take longer than others.

- Reducers may remain idle while waiting for Mapper output.
- Overall job execution time increases.
- Cluster resources are not utilized efficiently.
- The Hadoop job may experience timeouts or repeated failures.

Corrective Actions

- Divide input data into **balanced input splits**.
- Select an appropriate HDFS block size.
- Use proper file partitioning.
- Avoid excessive numbers of very small files.
- Use a suitable **custom partitioner** when data is highly skewed.
- Distribute large datasets more evenly.
- Monitor Mapper and Reducer task execution times.
- Use **Combiner functions** where appropriate to reduce the amount of intermediate data.

Conclusion

The problem is mainly caused by improper data distribution. Properly balanced input splits and partitioning can ensure that Mapper and Reducer tasks receive a more equal workload, improving Hadoop job performance.

2. HDFS Data Becomes Unavailable After a Single Node Failure

Problem Analysis

HDFS is designed to provide fault tolerance by storing multiple copies of data blocks on different DataNodes. This feature is called **replication**. If a DataNode fails, another DataNode containing a replica should continue serving the data.

If data becomes unavailable after only one node fails, the HDFS deployment probably has an incorrect replication or NameNode design.

For example, if the replication factor is set to **1**, each data block has only one copy. If that DataNode fails, the block becomes unavailable.

Another possible problem is the absence of **NameNode High Availability (HA)**. If the only NameNode fails, clients may be unable to access HDFS metadata even though the DataNodes are still running.

Probable Design Errors

- Replication factor is too low.
- Data blocks are not distributed across multiple DataNodes.
- Replicas are placed on the same failure domain.
- No NameNode High Availability.
- Poor rack-aware replica placement.
- DataNode health is not monitored.

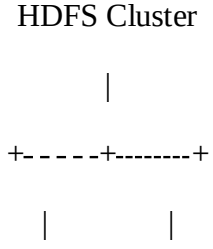
Effects

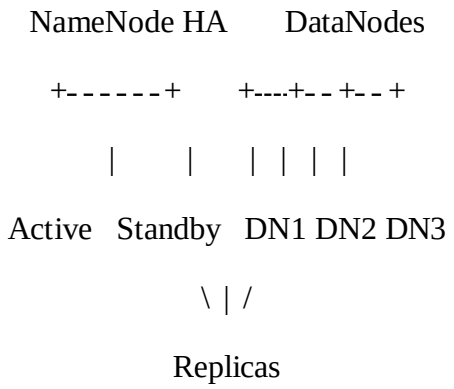
- Files become temporarily or permanently inaccessible.
- Applications using HDFS may fail.
- Data processing jobs may stop.
- A single hardware failure can affect the entire system.

Corrective Actions

1. Set an appropriate HDFS replication factor, commonly **3** for many production environments.
2. Store replicas on different DataNodes.
3. Use **rack awareness** so replicas are distributed across different racks where appropriate.
4. Configure **NameNode HA** with:
    - Active NameNode
    - Standby NameNode
5. Monitor DataNode health.
6. Regularly check HDFS replication status.
7. Replace failed DataNodes and allow HDFS to automatically re-replicate missing blocks.

Example Architecture





Conclusion

The main reason for data unavailability is insufficient fault tolerance. Proper block replication combined with NameNode HA can allow the HDFS system to continue operating even when a single node fails.

3. Apache NiFi ETL Workflow Creates Duplicate Records

Problem Analysis

Apache NiFi is commonly used to collect, transform, route, and load data into analytics systems. Duplicate records can occur when the same FlowFile or message is processed more than once.

For example, a data source may send the same file again, or a NiFi processor may retry an operation after a temporary failure. If the destination system does not recognize that the record has already been inserted, the same record may be stored multiple times.

Likely Causes

The major causes can include:

- The same source file is processed repeatedly.
- Incorrect NiFi flow configuration.
- Processor retries cause reprocessing.
- Network failures cause messages to be sent again.
- No unique identifier is assigned to records.
- The destination database does not perform duplicate checking.
- The ETL pipeline does not support idempotent operations.
- Failure handling and retry mechanisms are incorrectly configured.

Example

Suppose the source sends:

Student_ID Name

101 Arun

102 Ravi

103 Kiran

If the same FlowFile is processed twice, the analytics database may contain:

101 Arun

102 Ravi

103 Kiran

101 Arun

102 Ravi

103 Kiran

This produces incorrect analytical results.

Recommended Fixes

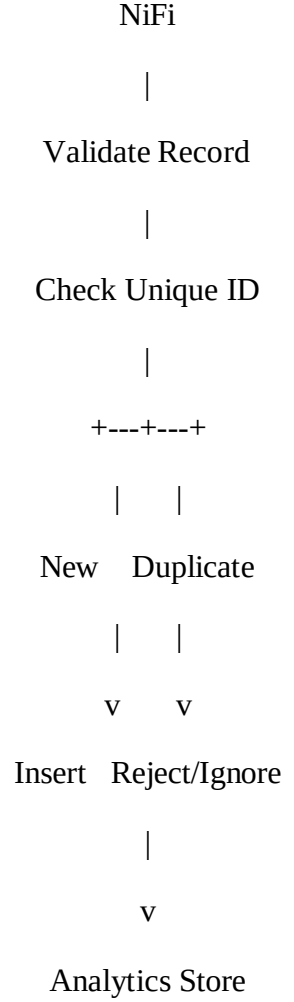
- Assign a **unique ID** to every record.
- Implement duplicate detection.
- Use proper NiFi FlowFile attributes.
- Configure retry mechanisms carefully.
- Ensure successful records are not processed again.
- Use database constraints such as unique keys.
- Use **upsert** operations instead of blindly inserting records.
- Design the pipeline to be **idempotent**.
- Monitor failed and retried FlowFiles.
- Maintain proper logging and provenance tracking.

Example Improved Flow

Data Source

|

v



Conclusion

Duplicate records generally result from repeated processing and inadequate duplicate detection. Using unique identifiers, idempotent processing, proper retry handling, and database constraints can maintain data accuracy.

4. YARN Cluster Suffers From Resource Starvation

Problem Analysis

YARN is responsible for managing resources in a Hadoop cluster. It allocates resources such as **CPU and memory** to applications.

When multiple users submit jobs at the same time, resource starvation may occur if one user or application consumes a large portion of the cluster resources. Other jobs then have to wait for resources.

For example, if a cluster has limited CPU and memory and one large application consumes most of the available resources, smaller applications may remain in the queue for a long time.

Likely Scheduling Error

The scheduling problem may be caused by:

- Improper scheduler configuration.
- No resource limits for users.
- Poor queue configuration.
- One application consuming excessive resources.
- Incorrect CPU or memory allocation.
- Lack of job prioritization.
- No fair resource-sharing mechanism.

Effects

- Some jobs experience long waiting times.
- Users may experience poor performance.
- Cluster resources are not fairly distributed.
- Important applications may be delayed.
- Overall cluster efficiency decreases.

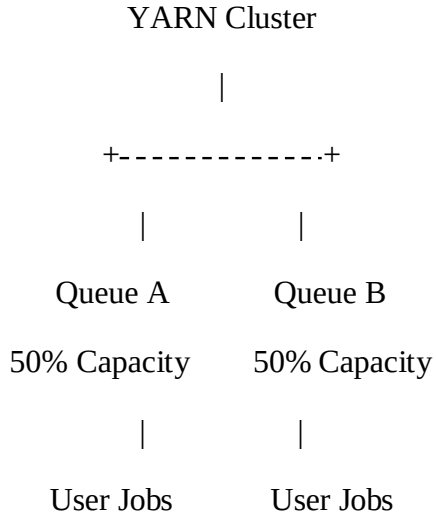
Recommended Resource Management Approach

A suitable YARN scheduler should be configured.

Capacity Scheduler

The **Capacity Scheduler** can divide cluster resources among different queues.

Example:



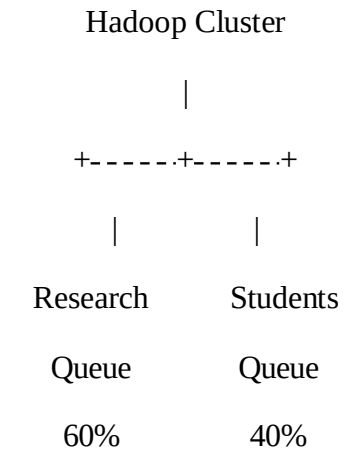
This ensures that different groups or departments receive a predefined amount of cluster capacity.

Other Corrective Actions

- Configure separate queues for different users.
- Set CPU and memory limits.
- Configure maximum resources per application.
- Set job priorities.
- Monitor resource consumption.
- Prevent a single application from consuming all resources.
- Use appropriate scheduling policies.
- Adjust queue capacity according to workload requirements.

Example

A university Hadoop cluster can be divided into:



This allows both groups to receive resources even when many jobs are submitted.

Conclusion

Resource starvation is mainly a scheduling and resource-allocation problem. Proper queue management, resource limits, and a suitable YARN scheduler can provide fair and efficient resource utilization.

5. Backup Restores Outdated Data After Recovery

Problem Analysis

Distributed storage systems require reliable backup and recovery mechanisms. If a system restores outdated information after a failure, the backup strategy is not properly maintaining the latest consistent version of the data.

For example, suppose the original data was updated on Monday, Tuesday, and Wednesday. If the latest successful backup was created on Monday and the system fails on Wednesday, recovery may restore Monday's data. The Tuesday and Wednesday changes would be missing.

Probable Design Flaws

- Backup is performed too infrequently.
- Latest backups are not verified.
- Replication is delayed.
- Replicas are not synchronized.
- Incremental backups are not properly configured.
- Backup jobs fail without being noticed.
- Old backup versions are used during recovery.
- Restore procedures are not regularly tested.

Effects

- Recent data may be lost.
- Analytics results may become incorrect.
- Users may receive outdated information.
- Business operations can be affected.
- Data consistency problems may occur.

Corrective Actions

1. Perform Regular Backups

Backups should be scheduled according to the importance and frequency of data changes.

2. Use Incremental Backups

Instead of copying everything each time, incremental backups store changes made after the previous backup.

3. Maintain Multiple Backup Versions

Keeping multiple versions allows administrators to restore an appropriate recovery point.

4. Verify Backup Completion

The system should check whether every backup has completed successfully.

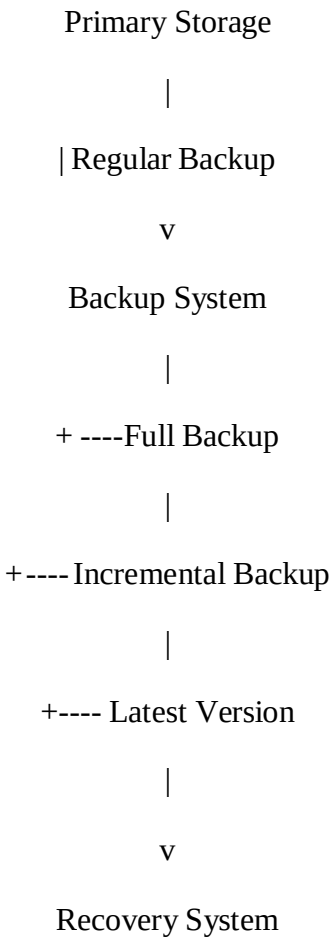
5. Monitor Replication

Replication should be monitored to ensure that replicas contain recent data.

6. Test Recovery

Backup systems should be tested regularly to confirm that the stored data can actually be restored.

Example



Conclusion

Restoring outdated data indicates that the backup or replication strategy is not maintaining a sufficiently recent and consistent copy. Frequent backups, incremental backups, replication monitoring, version management, and regular recovery testing can reduce data loss.